



(/rol/app/)

Help

Korean

검색

Seunghe...

Sites

홈 (/rol/app/) (home) > Observing OpenShift Service Mesh > Chapter 1: Observing OpenShift Service Mesh

Observing OpenShift Service Mesh

Lesson

Lab Environment

Guided Exercise: Tracing Services With Kiali, Tempo and OpenTelemetry

Visualize service mesh topology and analyze traffic flow by using the OpenShift Service Mesh traffic graph.

Inspect distributed traces from multiple interfaces to understand service interactions and identify performance bottlenecks.

Correlate metrics and traces to troubleshoot service performance issues.

Outcomes

- Visualize service mesh traffic topology with the Red Hat OpenShift console.
- Analyze application metrics and correlate them with distributed traces.
- Inspect distributed traces to understand service interactions and performance.

As the student user on the workstation machine, use the lab command to prepare your environment for this exercise, and to ensure that all required resources are available.

```
[student@workstation ~]$ lab start meshobservability-tracing
```

In this exercise, you explore the tracing observability features of Red Hat OpenShift Service Mesh (OSSM) by using the OpenShift web console with the OSSM Console plug-in (OSSMC). You use the BookInfo sample application, which consists of the following services:

productpage

Calls the details and reviews services to populate the page.

details

Contains book information.

reviews

Contains book reviews.

It also calls the ratings service. There are three versions of this service:

- reviews-v1 does not call the ratings service.
- reviews-v2 calls the ratings service and shows ratings as 1 to 5 black stars.
- reviews-v3 calls the ratings service and shows ratings as 1 to 5 red stars.

ratings

Contains book ranking information that accompanies a book review.



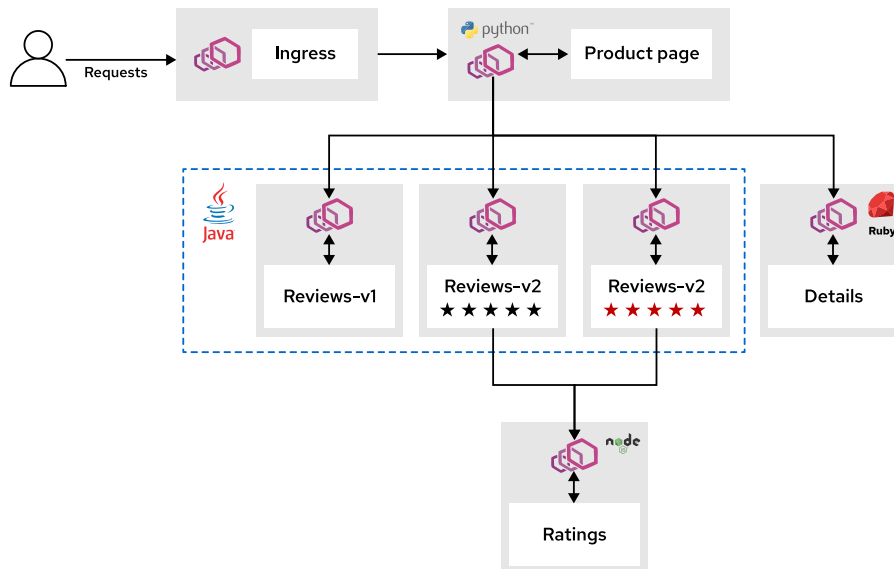


Figure 1.42: The BookInfo application components

The BookInfo application is already deployed in the `meshobservability-tracing` namespace with Istio sidecar proxies injected. A traffic generator continuously sends requests to the application to generate telemetry data. Mesh activity shows in the console a few minutes after the `lab` command finishes.

Instructions

1. Log in to the OpenShift cluster and confirm the BookInfo application deployment.
 - 1.1. From the workstation machine, open a terminal and log in to the OpenShift cluster as the `developer` user:

```
[student@workstation ~]$ oc login -u developer -p developer \
https://api.ocp4.example.com:6443
Login successful.
...output omitted...
```

Then, change to the `meshobservability-tracing` project:

```
[student@workstation ~]$ oc project meshobservability-tracing
...output omitted...
```

- 1.2. Verify that the BookInfo application pods are running in the `meshobservability-tracing` namespace:

```
[student@workstation ~]$ oc get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-5c4b4b459f-rs9fv         2/2     Running   0           15m
kiali-traffic-generator-r56xm        2/2     Running   0           15m
productpage-v1-6b7fcb56b-dk8f6       2/2     Running   0           15m
ratings-v1-7b76484f47-th2nk         2/2     Running   0           15m
reviews-v1-f8bd69dff-s9lxj           2/2     Running   0           15m
reviews-v2-64df7bdcbf-k2dfl          2/2     Running   0           15m
reviews-v3-5bf99fd74c-b2z26         2/2     Running   0           15m
```

All pods should show 2/2 in the `READY` column. This indicates that both the application container and the Istio sidecar proxy are running in each pod. The `kiali-traffic-generator` pod continuously sends requests to the `productpage` service to generate telemetry data.

2. Access the OpenShift web console.
 - 2.1. In the workstation machine, open the OpenShift web console in your browser.

Go to <https://console-openshift-console.apps.ocp4.example.com>, click `htpasswd_provider`. Use `developer` as the user and the password.

- 2.2. Change to the **Administrator** perspective.

The first time your `developer` user accesses the OpenShift console you see the **Developer** perspective. If prompted about the guided tour, then click **Skip tour** and change to the **Administrator** perspective.

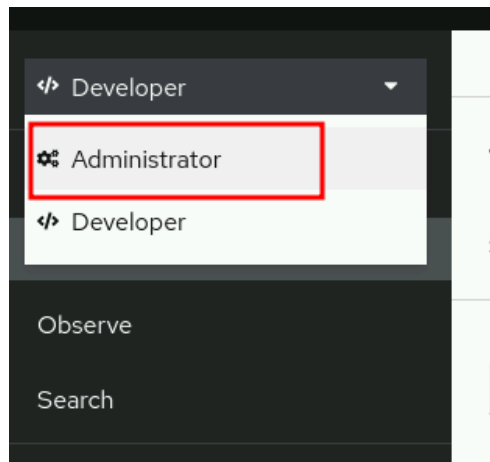


Figure 1.43: Menu for choosing the perspective in the OpenShift console

If you have previously accessed the OpenShift console, then you could already be in the Administrator perspective.

3. Explore the service mesh topology with the traffic graph, and inspect specific traces.

3.1. In the OpenShift web console, go to the **Service Mesh** → **Traffic Graph**.

3.2. Configure the Graph view to show the meshobservability-tracing namespace:

In the namespace selector at the top of the page, ensure that you select meshobservability-tracing.

The time range selector usually shows **Last 1m**. Ensure that you select a sufficient time range to see traffic data. Select **Last 5m**.

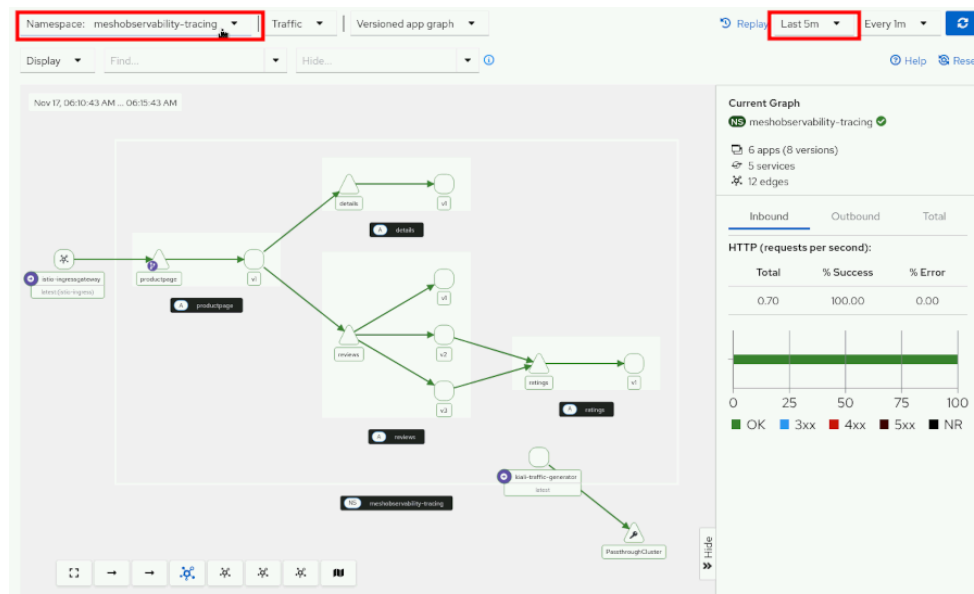


Figure 1.44: Namespace and time range selectors at the top of the Traffic Graph view

The graph can show yellow edges and nodes because during the start of the different services there were moments when they were unavailable.

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Your list of traces might be different. Adjust the time range to search for more traces.

3.3. Observe the service mesh topology:

The graph shows nodes representing the services in the BookInfo application and edges representing the traffic flow between them.

You should see:

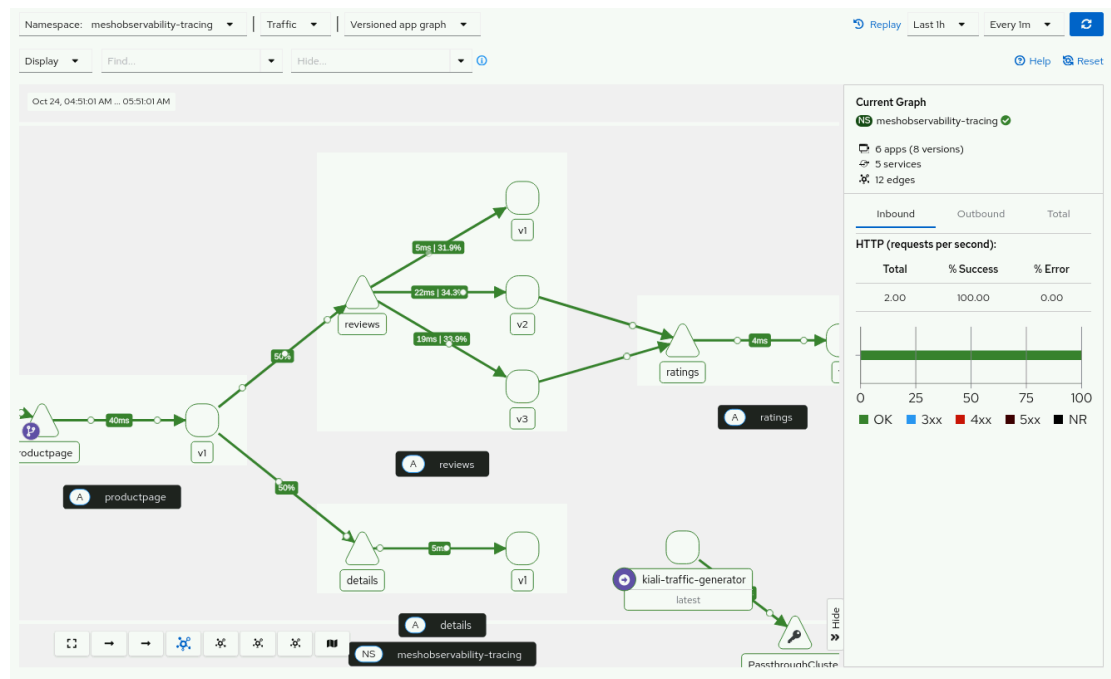
- The productpage service as the entry point
- Connections from productpage to details and reviews services
- Three versions of the reviews service
- A connection from reviews-v2 and reviews-v3 to the ratings service

3.4. Enable display options to visualize traffic distribution:

In the **Display** settings, enable the following display options:

- **Response Time**
- **Traffic Distribution**
- **Traffic Animation**

Use the scroll wheel on your mouse to increase the zoom level. When you increase the zoom level, the percentage of HTTP traffic on each edge is shown. The traffic distribution shows approximately equal distribution across the three versions of the reviews service.



3.5. Click the productpage triangle that represents the service.

Then, in the right side panel, click the **Traces** tab.

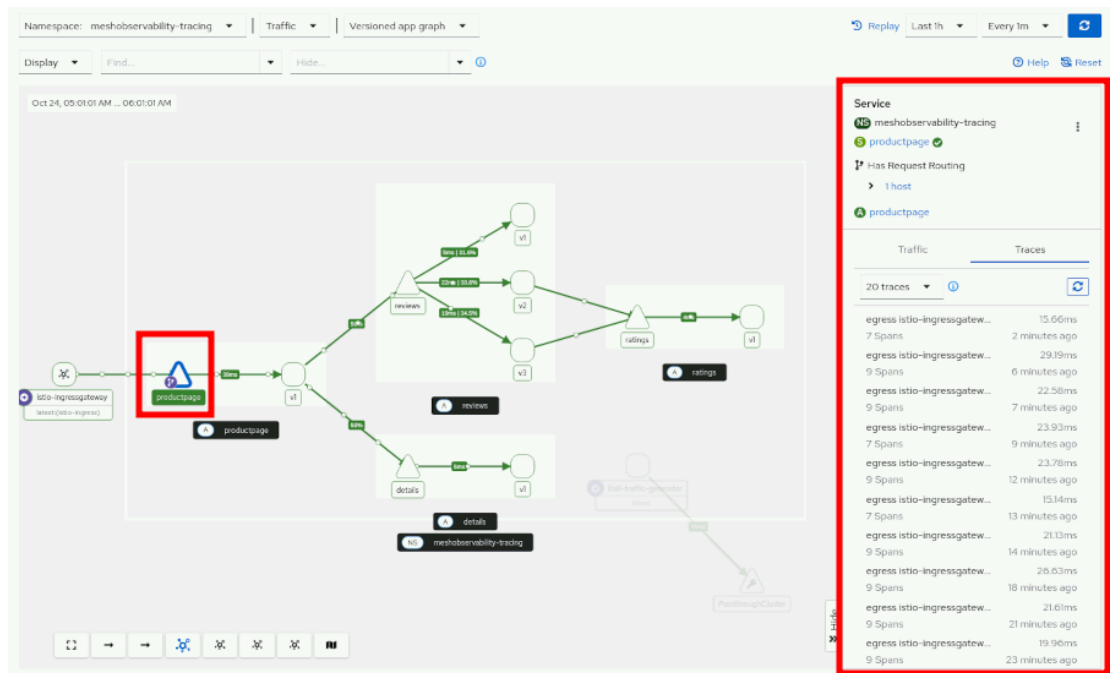


Figure 1.46: Right side panel showing a list of traces that use the product service

Note that there are some traces with seven spans and others with nine spans.

3.6. Click one of the traces with less than nine spans.

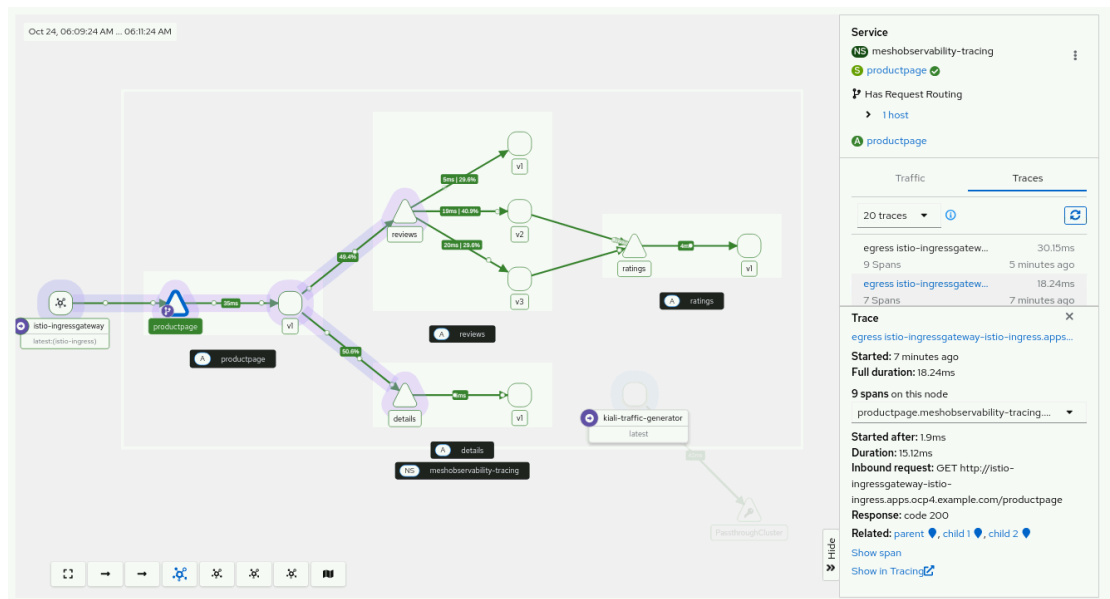


Figure 1.47: Traffic graph showing a bookinfo trace of seven spans

The traffic graph shows the flow of the traffic in that specific trace. Furthermore, a panel with all the information of that specific trace shows in the right side panel. Click the close button of that panel.

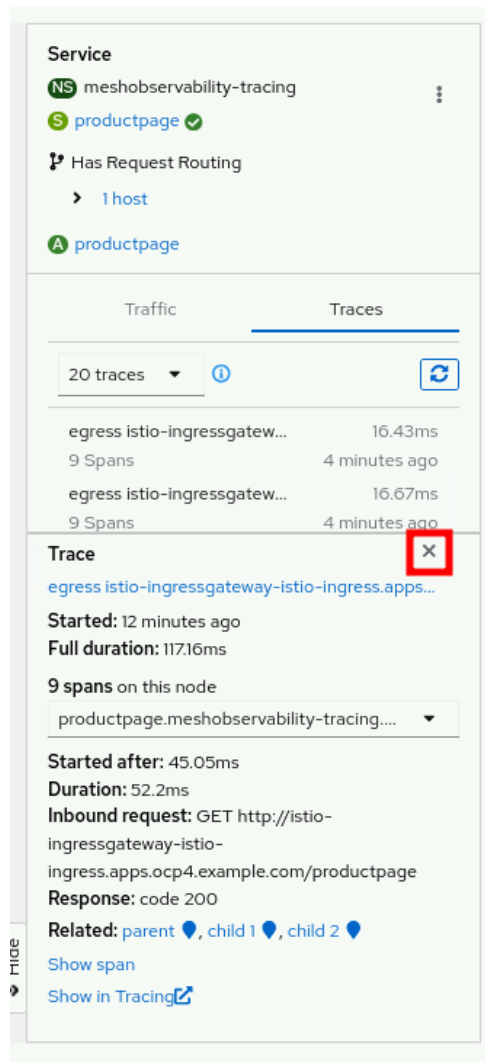


Figure 1.48: Right side panel showing details of the selected trace and the closing button

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Your list of traces might be different. Adjust the time range to search for more traces.

3.7. Click one of the nine spans traces.

The traffic graph shows the traffic flow that reaches the ratings service. Note that the panel with the trace information enables you to select each of the spans composing the trace, to see all the details.

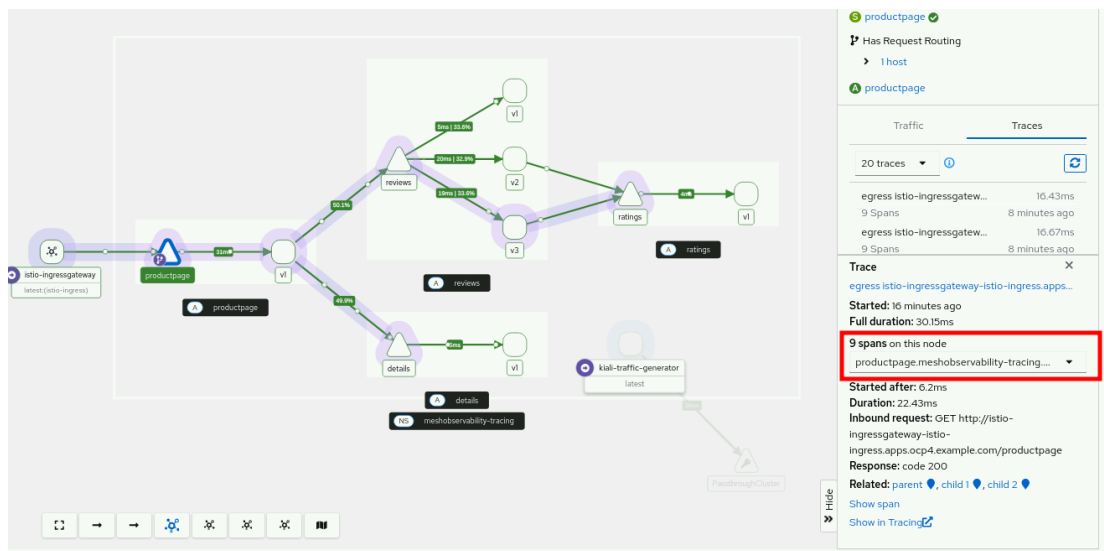


Figure 1.49: Traffic graph showing a bookinfo trace of nine spans with the span selector highlighted

4. Inspect distributed traces from the traffic graph.

4.1. In the workstation terminal, inspect the `rating-fault-injection.yaml` file to generate errors in the mesh.

Change to the `~/course/labs/meshobservability-tracing` directory, and read the file:

```
[student@workstation ~]$ cd ~/course/labs/meshobservability-tracing
[student@workstation meshobservability-tracing]$ cat rating-fault-injection.yaml
apiVersion: networking.istio.io/v1
kind: VirtualService 1
metadata:
  name: ratings-faults-vs
  namespace: meshobservability-tracing
spec:
  hosts:
  - ratings.meshobservability-tracing.svc.cluster.local 2
  http:
  - fault: 3
    delay:
      fixedDelay: 3s 4
      percentage:
        value: 50 5
    route:
    - destination:
        host: ratings.meshobservability-tracing.svc.cluster.local
        weight: 100
```

¹ Creation of a virtual service to inject faults.

² Host that applies the virtual service.

³ Type of HTTP traffic modification.

⁴ Delay applied to requests.

⁵ Percentage of requests that have the delay.

4.2. Apply the virtual service with the fault injection.

```
[student@workstation meshobservability-tracing]$ oc apply -f \
rating-fault-injection.yaml
virtualservice.networking.istio.io/ratings-faults-vs created
```

Wait for five or six minutes to see the traffic graph showing degraded services and edges.

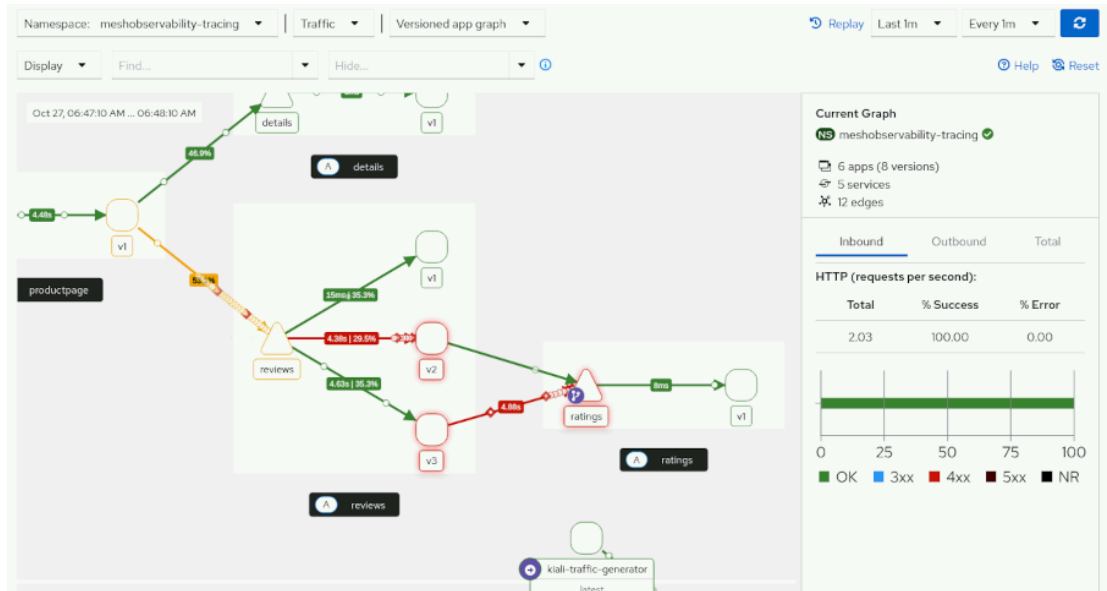


Figure 1.50: Traffic graph after applying the fault injection.

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Your list of traces might be different. Adjust the time range to search for more traces.

4.3. Locate spans with degradation, or errors.

The traffic animation shows degraded time responses in yellow, and errors in red.

Change the time range to ten or thirty minutes to get a larger list of spans, and then click the reviews triangle that represents the reviews service.

Then, click the Traces tab in the right side panel.

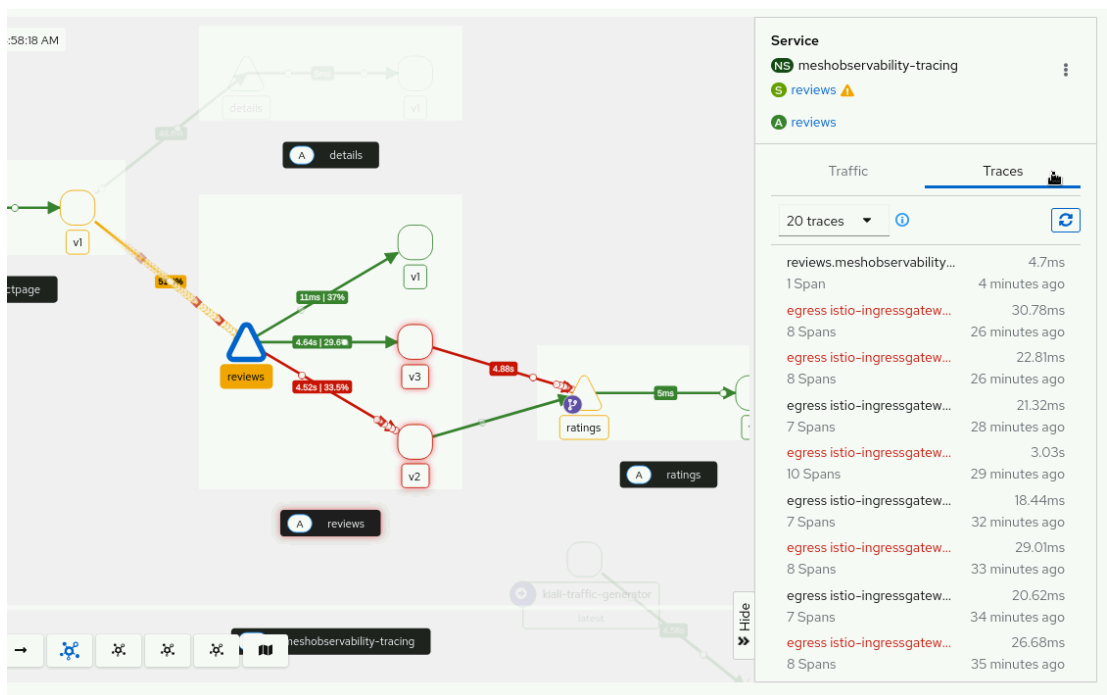


Figure 1.51: List of the reviews service spans

The Traces shows a list of traces that involve the reviews service, filtered from the overall trace data.

NOTE

If you need to refresh the trace list after changing the time range, then click the refresh button in the trace list.

This traces panel provides a service-centric view of distributed traces, showing only the traces where this specific service participated. Note that there are spans that spend a few milliseconds, but others can spend more than two seconds.

The graph visualization and trace data are integrated:

- You can start from the topology view and investigate traces for a specific service.
- This helps you investigate performance issues by first identifying problematic services in the graph, and then examining their specific trace patterns.

4.4. Click one of the traces that spends more than two seconds to inspect its details.

The right side panel shows another panel with the details of the trace:



Figure 1.52: Traffic graph showing a trace with errors

NOTE

Your graph might look different. Adjust graph parameters, such as traffic metrics per refresh, refresh interval, and graph zoom level, as needed. Your list of traces might be different. Adjust the time range to search for more traces.

4.5. In the details of the trace in the right side panel scroll down and click **Show span**.

The Traces tab within the Service Mesh tab, which is present in the OpenShift services view, opens.



Figure 1.53: Trace details in the service view

4.6. Inspect the failing span to find the errors in the logs.

Scroll down to see the **Span Details** table. There is one span with errors. Click the overflow menu of the span, and click **Logs**.

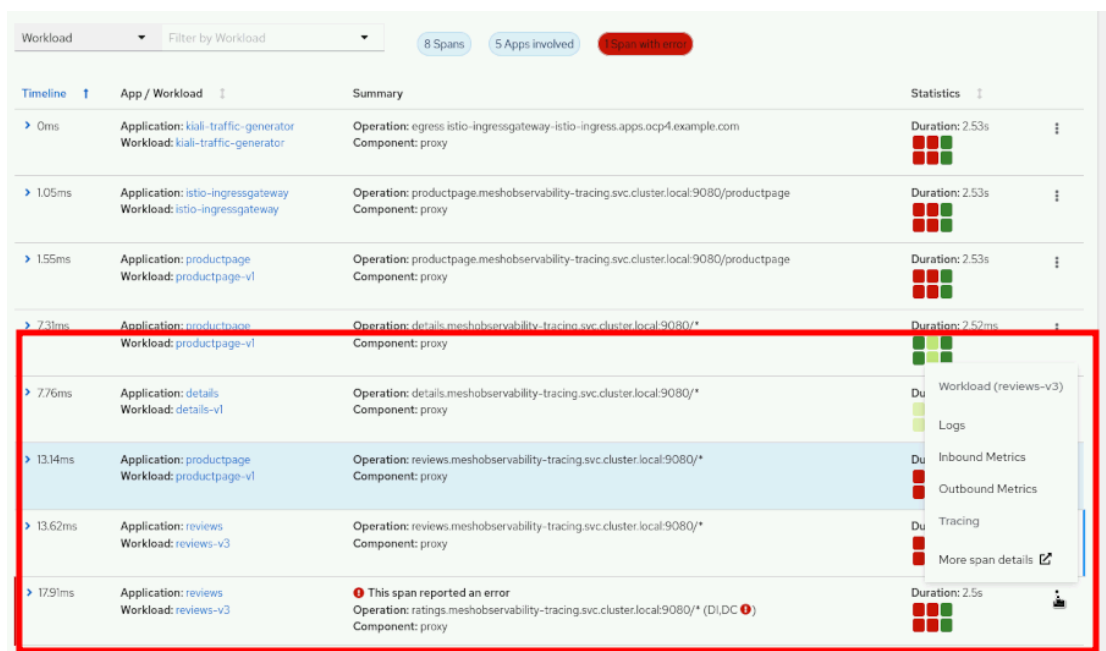


Figure 1.54: Menu to inspect a specific span

If your selected trace has more than one failing span, then choose the one at the bottom of the list.

The logs view for the `reviews` service pod opens showing the container log that emitted the error. In this case you see `java.net.SocketTimeoutException` exceptions when trying to connect to the `ratings` service.

5. Inspect distributed traces in the OpenShift console observe section.

5.1. Access the Observe menu:

Ensure that you are in the **Administrator** perspective. From the left navigation menu, click **Observe** → **Traces**. In the **Tempo** instance dropdown at the top, select `tracing-system/sample`. Leave the north tenant selected.

5.2. Review the Traces view:

The traces view shows a list of distributed traces collected from the `BookInfo` application.

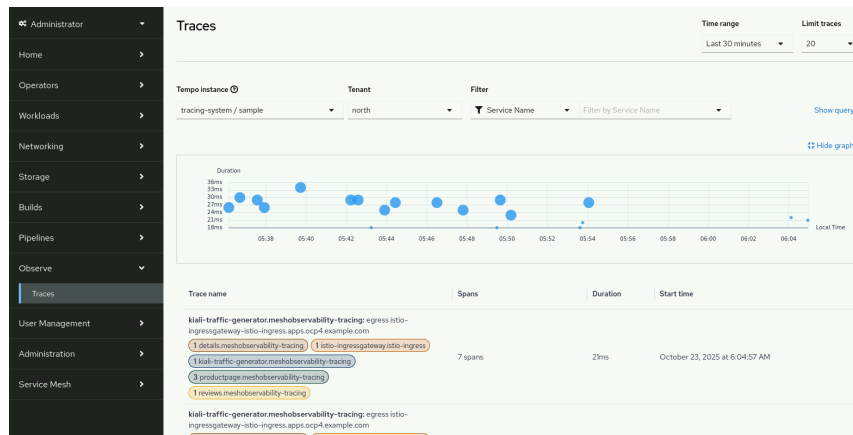


Figure 1.55: Traces view with a list of traces, including columns for trace name, number of spans, duration, and start time.

5.3. Examine the trace list:

All the traces start on the traffic generator component. Review the information shown for each trace:

- Trace name: Includes services involved in the trace
- Spans: Number of spans in the trace
- Duration: Total time for the request to complete
- Start Time: When the request was initiated

Look for traces with longer durations.

5.4. Select and analyze a trace:

Click the name of one of the traces in the list to view its details. The trace detail view opens, showing a timeline visualization of the trace spans.



Figure 1.56: Trace detail view with a timeline of spans, showing as horizontal bars

The timeline shows the following information:

- Each span as a horizontal bar
- The duration of each span, represented by the length of the bar
- Parent-child relationships between spans, indicated by indentation or tree structure
- The sequence and timing of service calls

5.5. Analyze the trace structure:

Observe the flow of the request through the BookInfo application:

The trace starts with a span for the incoming request from the traffic generator to productpage. You should see child spans showing calls from productpage to:

- details service
- reviews service: one of v1, v2, or v3.

If the trace hit reviews-v2 or reviews-v3, then you should see an additional span showing the call from reviews to the ratings service. Note whether these calls happen in parallel or sequentially, and which services contribute most to the total response time.

Note that the span hierarchy shows the parent-child relationship. For example, the productpage span is the parent, and the details and reviews spans are children of the productpage span.

Finish

On the workstation machine, use the lab command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish meshobservability-tracing
```

Revision: ad0007l-3.1-ed3-20260313-9c0e3a4



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f2000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책 (<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침 (<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정